

Week 3 Exercises

Nischal Pant

May 19, 2022

Please complete all exercises below. You may use any library that we have covered in class UP TO THIS POINT.

1) Two Sum - Write a function named `two_sum()`

Given a vector of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9` Output: `[0,1]` Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`. Example 2:

Input: `nums = [3,2,4]`, `target = 6` Output: `[1,2]` Example 3:

Input: `nums = [3,3]`, `target = 6` Output: `[0,1]`

Constraints:

`2 <= nums.length <= 104` `-109 <= nums[i] <= 109` `-109 <= target <= 109` Only one valid answer exists.

Note: For the first problem I want you to use a brute force approach (loop inside a loop)

The brute force approach is simple. Loop through each element x and find if there is another value that equals to $target - x$

Use the function `seq_along` to iterate

```
# we will define the function called two_sum (num_vector = our number vector,
# target = the number we want). These will be our arguments.
two_sum <- function(nums_vector, target) {

  # we are going create a loop where, i is used through indices as values in
  # the num_vector, using the seq_along function
  for (i in seq_along(nums_vector)) {

    # then in the second loop, j will take the value in num_vector, using
    # seq_along function as well. It is a loop within a loop.
    for (j in seq_along(nums_vector)) {

      # we create a condition that if i is not equal to j & if i and j add up
      # to our target, it will print out the output. Using the print function
      # to simply print the output with the indices threw many errors and made
      # it look weird as well
```

```

    if (i != j & nums_vector[i] + nums_vector[j] == target) {
      cat("[1] ", i, " ", j, "\n", sep = "")
    }
  }
}
}

# now we test the code by creating the vector which we will call upon using
# our function we created
nums_vector <- c(5, 7, 12, 34, 6, 10, 8, 9)
target <- 13

# using our custom function we call the vector and our target and put them
# through the loop

two_sum(nums_vector, target)

## [1] 1 7
## [1] 2 5
## [1] 5 2
## [1] 7 1

# now our output show all the indices where the two values add up to the target
# as shown on the expected answers

```

- 2) Now write the same function using hash tables. Loop the array once to make a hash map of the value to its index. Then loop again to find if the value of target-current value is in the map.

The keys of your hash table should be each of the numbers in the `nums_vector` minus the target.

A simple implementation uses two iterations. In the first iteration, we add each element's value as a key and its index as a value to the hash table. Then, in the second iteration, we check if each element's complement ($\text{target} - \text{nums_vector}[i]$) exists in the hash table. If it does exist, we return current element's index and its complement's index. Beware that the complement must not be `nums_vector[i]` itself!

```

# first, let's load the hash package onto R
library(hash)

## Warning: package 'hash' was built under R version 4.2.3
## hash-2.2.6.2 provided by Decision Patterns

# we will now define the function called two_sum (num_vector = our number
# vector, target = the number we want). These will be our arguments.
two_sum <- function(nums_vector, target) {

  # we create a blank hash table to store our variables for the function.
  h <- hash()

  # for the sake of making the code work I will be using two iterations.
  # in the first for loop, i will be going through all the values in num_vector
  # using the seq_along function in which we will create a subset of values
  # equal to the target - nums_vector which is the key we will be looking for
  # in the next loop (complement_key)
  for (i in seq_along(nums_vector)) {
    complement <- target - nums_vector[i]
    complement_key <- as.character(complement)
  }
}

```

```

# now for the second iteration we check if an element of the complement_key
#already exists in our hash table and if it does we add the indices to the
#complement_index
if (complement_key %in% keys(h)) {
  complement_index <- h[[complement_key]]
  cat("[1] ", complement_index, " ", i, "\n", sep = "")
}

# and finally in this code we return current element's index and its
# complement's index to the hash table (mapping)
h[[as.character(nums_vector[i])]] <- i
}
}

# now we test the code by creating the vector which we will call upon using
# our function we created
nums_vector <- c(5, 7, 12, 34, 6, 10, 8, 9)
target <- 15

# using our custom function, we call the vector and our target and put them
# through the loop and searching the hash table we get this.
two_sum(nums_vector, target)

## [1] 1 6
## [1] 2 7
## [1] 5 8

```